



Task Examples

Use the left navigation to view one task example from each intent.

1 Example Task 5368 Root Case Analysis

Example Task 5368

Name: liam-hq-liam-1408-rca
Intent: Root Case Analysis
Difficulty: Medium
Language: TypeScript

Repo

Repository: <https://github.com/liam-hq/liam>
Checkout (commit): 21113d02f61b166a8a0d2316ff7b1566fc84c551

Prompt

Some UNIQUE constraints from my pg_dump file aren't reflected in the SQL parser's column metadata. The parser's own one-to-one test passes, so the feature works for some patterns. What determines which UNIQUE constraints get picked up and which don't?

LLM Prompt Review

Prompt QC Feedback

****Grading Item 1**** (1/1)

The question seeks nuanced insight into why certain UNIQUE constraints are missed, indicating more than a superficial query.

****Grading Item 2**** (1/1)

It reads as a natural developer-to-developer question without role-play language.

****Grading Item 3**** (1/1)

It avoids basic definitions and instead probes behavior of a specific feature.

****Grading Item 4**** (1/1)

Answering requires understanding how the parser processes repo-specific inputs, implying real exploration.

****Grading Item 5**** (1/1)

The prompt is a question only and does not include pasted code or logs.

****Grading Item 6**** (1/1)

It is specific and asks a concrete question about repo behavior with a verifiable answer.

****Grading Item 7**** (1/1)

The prompt aligns with the intent by demanding reasoning about repository-driven functionality.

****Grading Item 8**** (1/1)

It calls for technical judgment about parser behavior rather than a trivial lookup.

****Grading Item 9**** (1/1)

No meta-instructions or formatting directives are present.

****Grading Item 10**** (1/1)

Contains only ASCII characters and standard punctuation.

FINAL_SCORE: 10

Rubric

id	tag	description	weight	rationale	dependent_on
48552	reasoning	Identifies <code>handleOneToOneRelationships</code> as the shared utility both parsers call, which checks <code>column.unique</code> to determine cardinality.	bonus	The shared utility is the pivot point. The discrepancy must be upstream of it.	
48553	reasoning	Explains the <code>schema.rb</code> parser sets <code>column.unique</code> through <code>extractColumnOptions</code> when the column definition includes <code>unique: true</code> .	critical	Understanding WHY <code>schema.rb</code> works is half of explaining the discrepancy.	

48554	reasoning	Explains the SQL parser only sets column.unique from inline CONSTR_UNIQUE in ColumnDef elements within handleCreateStmt.	critical	Understanding what the SQL parser DOES handle narrows the gap.	
48555	reasoning	Identifies ALTER TABLE ADD CONSTRAINT UNIQUE is dropped by the SQL converter without updating column.unique.	critical	The most impactful gap since pg_dump uses this pattern.	
48556	reasoning	Explains pg_dump separates constraints into ALTER TABLE statements instead of inlining them in column definitions.	bonus	The root cause is a format mismatch.	
48557	reasoning	Notes the SQL parser's one-to-one test uses inline UNIQUE on the column definition.	critical	The prompt explicitly asks why the test suite doesn't catch this.	48554
48558	reasoning	Notes CREATE UNIQUE INDEX does not result in column.unique being set to true.	critical	A second distinct gap in the SQL parser's uniqueness handling.	
48559	reasoning	Notes constraintToRelationship checks contype against CONSTR_FOREIGN and returns undefined for all other constraint types.	bonus	Traces the exact code that discards UNIQUE constraints.	

48560	reasoning	Notes handleCreateStmt only processes elements where ColumnDef in elt, skipping table-level Constraint nodes entirely.	bonus	A third gap.	
48561	reasoning	Notes the ALTER TABLE variant of the one-to-one test also places UNIQUE inline on the column in CREATE TABLE, not in the ALTER TABLE statement.	critical	Shows both SQL one-to-one tests exercise the same path.	48557
48562	reasoning	Notes schema.rb one-to-one detection depends on inline column-level `unique: true`; unique index declarations are stored as indexes and do not by themselves set `column.unique`.	critical	Explains the architectural reason the schema.rb parser works.	
48563	reasoning	Claims both parsers have the same uniqueness handling bug.	penalty	The schema.rb parser handles inline unique: true correctly.	
48564	reasoning	Claims handleOneToOneRelationships is the root cause rather than the upstream uniqueness handling in the SQL converter.	penalty	handleOneToOneRelationships correctly reads column.unique.	

48565	reasoning	Attributes the issue to pr.patch changes rather than pre-existing converter behavior.	penalty	The SQL converter code at the base commit already has these gaps.	
48566	completeness	Locates the SQL converter at frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts.	critical	File path grounds the SQL parser analysis.	
48567	completeness	Locates the schema.rb parser at frontend/packages/db-structure/src/parser/schemarb/parser.ts.	critical	File path grounds the schema.rb parser analysis.	
48568	completeness	Locates handleOneToOneRelationships at frontend/packages/db-structure/src/parser/utls/handleOneToOneRelationships.ts.	critical	File path grounds the shared utility.	
48569	completeness	Identifies extractColumnOptions as the schema.rb parser function that reads the unique keyword from column definitions.	critical	Names the specific function in the schema.rb parser responsible for the working path.	
48570	completeness	Identifies constraintToRelationship as the SQL converter function that filters out non-FK constraints.	critical	Names the specific function where UNIQUE constraints are discarded.	
48571	completeness	Notes no SQL parser test exercises the scenario where uniqueness is added via ALTER TABLE rather than inline on the column.	critical	The prompt explicitly asks why the test suite doesn't catch this.	

48572	completeness	Notes the FIXME comment and skip'd test in the SQL parser test file about CONSTRAINT statement support.	critical	Documentary evidence the parser creators knew about the limitation.	
48573	completeness	Locates the SQL parser test file at frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts.	bonus	File path grounding.	
48574	completeness	Provides concrete SQL examples showing the inline pattern that works versus the ALTER TABLE pattern that fails.	bonus	Examples make the gap tangible.	
48575	completeness	Notes table-level UNIQUE constraints in CREATE TABLE are also skipped because handleCreateStmt only processes ColumnDef elements.	bonus	A third distinct gap.	
48576	completeness	Notes handleIndexStmt stores index data in table.indexes without updating column.unique.	bonus	Clarifies index data exists but is disconnected from column metadata.	
48577	completeness	Claims the schema.rb parser also fails to detect column uniqueness from inline unique: true.	penalty	The schema.rb parser handles this correctly.	

48578	completeness	Claims inline UNIQUE in CREATE TABLE SQL is also broken.	penalty	Inline column-level UNIQUE is the one path the SQL parser handles correctly.	
48579	style	Traces both parsers systematically, comparing their uniqueness handling paths.	bonus	A cross-parser comparison is clearer when both sides are traced explicitly.	
48580	style	References specific file paths, function names, and code patterns from the repository.	bonus	Repo-grounded answers are more verifiable.	
48581	style	Avoids proposing a complete code patch, staying focused on diagnosis.	bonus	RCA intent is diagnosis, not implementation.	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: reasoning, completeness, and style. All three categories are present. (2/2)**

All three required categories are present as separate top-level arrays in the rubric JSON.

****Grading Item 2:** The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)**

There are 30 total items (14 reasoning, 13 completeness, 3 style), which is within the suggested range and offers broad coverage for the task.

****Grading Item 3:** Every grading item is atomic and self-contained. (2/2)**

Each item focuses on a single, distinct expectation; none bundle multiple independent checks.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)****

The descriptions cite concrete behaviors, file paths, or statements that can be objectively verified.

****Grading Item 5: Every grading item is derivable from the provided golden_answer. (1/2)****

Nearly all items trace directly to details in the golden answer, but the penalty about attributing the issue to PR changes lacks support from the golden answer, so full credit isn't warranted.

****Grading Item 6: The rubric is comprehensive relative to the golden_answer. (2/2)****

All major points from the golden answer—shared utility, schema.rb handling, SQL converter gaps, pg_dump patterns, and test coverage—are represented.

****Grading Item 7: The rubric would award full marks to the provided golden_answer. (2/2)****

The golden answer satisfies every critical requirement and avoids all penalties described, so it would earn full credit.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths. (2/2)****

Although detailed, the items allow any repo-grounded explanation matching the documented behaviors, not just the golden answer's wording.

****Grading Item 9:** The reasoning category evaluates correctness of reasoning, technical judgment, conceptual understanding, and justification. (2/2)**

Reasoning items focus on how uniqueness is set or missed, why tests pass, and relevant parser mechanics—true reasoning content.

****Grading Item 10:** The completeness category evaluates coverage of all major question parts and necessary supporting details. (2/2)**

Completeness items emphasize referencing key files, tests, and evidence, aligning with coverage requirements rather than correctness.

****Grading Item 11:** The style category evaluates clarity, organization, readability, and professionalism of the response. (2/2)**

Style items focus on presentation aspects such as staying diagnostic, citing repo paths, and tracing both parsers coherently.

****Grading Item 12:** Grading items are placed in appropriate categories. (2/2)**

Each item's placement matches its focus; there are no misplaced reasoning/completeness/style checks.

****Grading Item 13:** The rubric includes the task-wide expectations: reasoning quality, completeness, and code style. (2/2)**

Each of the three categories contains items covering these broad expectations.

****Grading Item 14: The rubric includes intent-specific grading items when required. (2/2)****

Given the RCA intent, items about root causes, parser gaps, and test explanations satisfy the intent-specific needs.

****Grading Item 15: Weights use only valid values and are well calibrated. (2/2)****

All items use `critical`, `bonus`, or `penalty`, and their assignment reflects the importance of each expectation.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior. (2/2)****

Penalties address incorrect attributions or conflating parser behaviors, each representing distinct harmful misunderstandings.

****Grading Item 17: Dependency usage is logically sound. (2/2)****

Dependencies exist only where logical prerequisites are needed, and their directions make sense.

****Grading Item 18: The rubric is not overprescriptive. (2/2)****

While detailed, it allows alternative repo-grounded explanations and doesn't force an exact narrative sequence.

Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)

Items are non-redundant and each adds evaluative value.

FINAL_SCORE: 37

Golden Answer

The shared cardinality logic

Both the schema.rb and PostgreSQL SQL parsers call the same `handleOneToOneRelationships` function from `frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`. This function iterates every relationship and promotes it from `ONE_TO_MANY` to `ONE_TO_ONE` when the foreign key column has `column.unique === true`. No other code path touches cardinality. So the discrepancy comes down to whether each parser sets `column.unique` before this function runs.

How the schema.rb parser sets column.unique

In the schema.rb parser (`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to `extractColumnOptions`, which reads keyword arguments from the column definition. When a column is declared with `unique: true` in the schema.rb DSL:

```
```ruby
t.bigint "user_id", unique: true
```
```

The `case 'unique'` branch sets `column.unique = value instanceof TrueNode`. This is a direct, inline property on the column definition. For this inline schema.rb pattern, the uniqueness appears right where the parser expects it; separate `t.index ..., unique: true` declarations are stored as indexes, not copied into `column.unique`.

How the SQL parser fails to set column.unique

The PostgreSQL SQL parser

(`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`) handles column uniqueness inside `handleCreateStmt`. It checks each `ColumnDef` element for inline `CONSTR\_UNIQUE` or `CONSTR\_PRIMARY` constraints:

```
```typescript
unique:
 colDef.constraints
 ?.filter(isConstraintNode)
 .some((c) =>
 ['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(
 c.Constraint.contype ?? "",
),
) || false,
...
```
```

This works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE constraint is part of the column definition. But `pg\_dump` does not emit uniqueness this way. A standard `pg\_dump` output separates constraints from table definitions:

```
```sql
CREATE TABLE posts (
 user_id integer NOT NULL
);
```
```

```
ALTER TABLE ONLY posts ADD CONSTRAINT posts_user_id_key UNIQUE (user_id);
...
```
```

The `ALTER TABLE` statement goes through `handleAlterTableStmt`, which processes `AT\_AddConstraint` commands by calling `constraintToRelationship`. But that function returns early for anything except foreign keys:

```
```typescript
if (constraint.contype !== 'CONSTR_FOREIGN') {
  return ok(undefined)
}
...
```
```



The UNIQUE constraint is dropped. `column.unique` stays `false`.

The same gap exists for two other SQL patterns. Table-level UNIQUE constraints inside `CREATE TABLE` (`UNIQUE (user_id)` as a separate clause) are skipped because `handleCreateStmt` only processes elements where `'ColumnDef'` in `elt`. And `CREATE UNIQUE INDEX` statements are captured by `handleIndexStmt` as Index objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique` field.

### ## Why the inline schema.rb case avoids this problem

For the schema.rb path that drives one-to-one detection in these tests, uniqueness is detected when the column call itself includes `unique: true`, such as `t.bigint "user_id", unique: true`. The schema.rb parser reads that inline attribute through `extractColumnOptions`, which handles `unique` as a dedicated keyword.

Unique indexes in schema.rb, such as `t.index ["user_id"], unique: true`, are parsed as index metadata instead. They do not by themselves set `column.unique`, so they are not the schema.rb mechanism that `handleOneToOneRelationships` uses for cardinality.

The SQL format in the failing pg\_dump-style case has no equivalent inline column flag. PostgreSQL splits these apart: the column definition carries type and nullability, while constraints live in separate statements (either table-level in `CREATE TABLE` or via `ALTER TABLE`). The SQL parser was built to handle the inline form but not the separated form.

### ## Why the SQL parser's one-to-one test passes

The `foreign key (one-to-one)` test in `frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts` declares:

```
```sql
CREATE TABLE posts (
  id BIGSERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) UNIQUE
);
```
```

The UNIQUE is inline on the column definition, hitting the one path the SQL parser handles. The test never uses the separated form (``ALTER TABLE ADD CONSTRAINT UNIQUE`` or ``CREATE UNIQUE INDEX``). The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself. Because both tests exercise the single working path, they pass despite the gaps.

The test file also contains a skip'd test with a FIXME comment (``// FIXME: 'CONSTRAINT' statement is not supported yet``) for table-level CONSTRAINT syntax inside CREATE TABLE. The skip acknowledges the parser developers were aware of the limitation for table-level constraints.

## ## The root cause

The discrepancy is a format mismatch. The SQL parser's uniqueness detection was built for inline column constraints (the form used in the tests), but ``pg_dump`` emits uniqueness through ALTER TABLE and CREATE INDEX statements (which the parser drops). The schema.rb parser avoids this problem for inline column-level ``unique: true`` declarations because those set ``column.unique`` directly. The shared ``handleOneToOneRelationships`` utility is correct; it reads ``column.unique`` correctly. The issue is that the SQL parser's upstream code fails to populate that field for the most common PostgreSQL output patterns.

## Golden Answer Review

Golden Answer QC Feedback

**\*\*Grading Item 1: Internal consistency and technical accuracy (3/3)\*\***

No contradictions or unsupported claims; code references and logic align with repo context.

---

**\*\*Grading Item 2: Repo-grounded analysis (3/3)\*\***

References concrete files, functions, and behaviors within the repository.

---

**\*\*Grading Item 3: Direct answer to prompt (2/2)\*\***

Explains exactly which UNIQUE patterns are detected and why.

---

**\*\*Grading Item 4: Reasoning chain (3/3)\*\***

Walks through the causal steps from parser behavior to observed discrepancy.

---

**\*\*Grading Item 5: Technical precision (2/2)\*\***

Mechanistic descriptions specify how data flows and why gaps occur.

---

**\*\*Grading Item 6: Factual accuracy (2/2)\*\***

Statements about code and framework behavior are correct for this repo.

---

**\*\*Grading Item 7: Critical rubric requirements (3/3)\*\***

All must-have rubric elements are addressed.

---

**\*\*Grading Item 8: Penalty avoidance (2/2)\*\***

No penalty-triggering behaviors present.

---

**\*\*Grading Item 9: Comprehensiveness (2/2)\*\***

Covers all major aspects of the discrepancy and supporting evidence.

---

**\*\*Grading Item 10: Clarity for non-experts (2/2)\*\***

Clear structure and language suitable for onboarding readers.

---

**\*\*Grading Item 11: Concrete supporting details (2/2)\*\***

Includes file paths, function names, and specific SQL patterns.

---

**\*\*Grading Item 12: Intent-specific expectations (3/3)\*\***

Meets root-cause-analysis expectations, focusing on diagnosis.

---

**\*\*Grading Item 13: PR Triage structure (2/2)\*\***

Not applicable; full credit granted.

---

**\*\*Grading Item 14: ASCII-only (1/1)\*\***

No non-ASCII characters used.

---

FINAL\_SCORE: 32

## Validate Golden Answer

Golden Answer Validation

```
{
 "results": [
 {
 "id": "reasoning-48560",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Notes handleCreateStmt only processes elements where ColumnDef in elt, skipping table-level Constraint nodes entirely.",
 "rationale": "A third gap.",
 "dependent_on": [],
```

```

 "pass": true,
 "evidence": "\u201cTable-level UNIQUE constraints inside CREATE TABLE (`UNIQUE (user_id)` as a
separate clause) are skipped because `handleCreateStmt` only processes elements where `ColumnDef` in
elt`.\u201d",
 "reasoning": "The golden answer explicitly states that handleCreateStmt restricts processing to
elements with ColumnDef, explaining why table-level constraint nodes are skipped, directly satisfying the
criterion."
 },
 {
 "id": "reasoning-48557",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Notes the SQL parser's one-to-one test uses inline UNIQUE on the column definition.",
 "rationale": "The prompt explicitly asks why the test suite doesn't catch this.",
 "dependent_on": [
 "reasoning-48554"
],
 "pass": true,
 "evidence": "\u201cThe `foreign key (one-to-one)` test \u2026 declares \u2026 `user_id INT
REFERENCES users(id) UNIQUE`. The UNIQUE is inline on the column definition, hitting the one path the
SQL parser handles. The test never uses the separated form \u2026 Because both tests exercise the
single working path, they pass despite the gaps.\u201d",
 "reasoning": "The answer explicitly states that the SQL parser's one-to-one test places the
UNIQUE constraint inline on the column definition, explaining why the tests pass and showing
understanding of the test setup."
 },
 {
 "id": "reasoning-48558",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Notes CREATE UNIQUE INDEX does not result in column.unique being set to true.",
 "rationale": "A second distinct gap in the SQL parser's uniqueness handling.",
 "dependent_on": [],
 "pass": true,
 "evidence": "And `CREATE UNIQUE INDEX` statements are captured by `handleIndexStmt` as Index
objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique` field.",

```

"reasoning": "The golden answer explicitly states that CREATE UNIQUE INDEX statements do not cause column.unique to be set to true, highlighting this limitation in the SQL parser."

},

{

"id": "reasoning-48561",

"tag": "reasoning",

"weight": "critical",

"description": "Notes the ALTER TABLE variant of the one-to-one test also places UNIQUE inline on the column in CREATE TABLE, not in the ALTER TABLE statement.",

"rationale": "Shows both SQL one-to-one tests exercise the same path.",

"dependent\_on": [

"reasoning-48557"

],

"pass": true,

"evidence": "\u201cThe `foreign key (one-to-one)` test ... declares \u2026 UNIQUE is inline \u2026 The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself.\u201d",

"reasoning": "The golden answer explicitly states that the ALTER TABLE one-to-one test still places the UNIQUE constraint inline in the CREATE TABLE definition, demonstrating awareness that both tests exercise the same code path."

},

{

"id": "reasoning-48562",

"tag": "reasoning",

"weight": "critical",

"description": "Notes Rails schema.rb format always inlines column attributes on the column definition call as a design convention.",

"rationale": "Explains the architectural reason the schema.rb parser works.",

"dependent\_on": [],

"pass": true,

"evidence": "Rails' `schema.rb` dumper bakes column attributes directly into the column definition call. A unique column appears as `t.bigint `\"user\_id\", unique: true`, not as a separate constraint or index statement.",

"reasoning": "The answer explicitly states that Rails schema.rb inlines column attributes on the column definition call, matching the criterion."

},

```

{
 "id": "reasoning-48555",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Identifies ALTER TABLE ADD CONSTRAINT UNIQUE is dropped by the SQL converter
without updating column.unique.",
 "rationale": "The most impactful gap since pg_dump uses this pattern.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe `ALTER TABLE` statement goes through `handleAlterTableStmt`, which
processes `AT_AddConstraint` commands by calling `constraintToRelationship`. But that function returns
early for anything except foreign keys\u2026 The UNIQUE constraint is dropped. `column.unique` stays
`false`.\u201d",
 "reasoning": "The answer explicitly states that ALTER TABLE ADD CONSTRAINT UNIQUE statements
are processed but discarded because the converter returns early unless the constraint is foreign, meaning
column.unique is never updated."
},
{
 "id": "reasoning-48565",
 "tag": "reasoning",
 "weight": "penalty",
 "description": "Attributes the issue to pr.patch changes rather than pre-existing converter behavior.",
 "rationale": "The SQL converter code at the base commit already has these gaps.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The response explains the existing limitations of the SQL parser without attributing them
to any recent patch: \u201cThe SQL parser\u2019s uniqueness detection was built for inline column
constraints\u2026 pg_dump emits uniqueness through ALTER TABLE and CREATE INDEX statements
(which the parser drops).\u201d",
 "reasoning": "The answer attributes the issue to longstanding behavior in the SQL parser\u2019s
handling of constraints, not to any PR or new patch changes, so it meets the criterion."
},
{
 "id": "reasoning-48563",
 "tag": "reasoning",
 "weight": "penalty",

```

```

 "description": "Claims both parsers have the same uniqueness handling bug.",
 "rationale": "The schema.rb parser handles inline unique: true correctly.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cWhen a column is declared with `unique: true` \u2026 The `case 'unique` branch sets `column.unique = value instanceof TrueNode` \u2026 The PostgreSQL SQL parser \u2026 works for \u2026 inline \u2026 But `pg_dump` does not emit uniqueness this way \u2026 The UNIQUE constraint is dropped.\u201d",
 "reasoning": "The golden answer differentiates the parsers, explaining the schema.rb parser correctly handles inline unique attributes while the SQL parser fails for pg_dump patterns; it does not claim both share the same bug."
 },
 {
 "id": "reasoning-48564",
 "tag": "reasoning",
 "weight": "penalty",
 "description": "Claims handleOneToOneRelationships is the root cause rather than the upstream uniqueness handling in the SQL converter.",
 "rationale": "handleOneToOneRelationships correctly reads column.unique.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThis function iterates every relationship and promotes it from `ONE_TO_MANY` to `ONE_TO_ONE` when the foreign key column has `column.unique === true`. No other code path touches cardinality. So the discrepancy comes down to whether each parser sets `column.unique` before this function runs.\u201d",
 "reasoning": "The golden answer explicitly states that handleOneToOneRelationships functions correctly and attributes the discrepancy to upstream SQL parser logic failing to set column.unique, thus avoiding the prohibited claim."
 },
 {
 "id": "reasoning-48552",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Identifies handleOneToOneRelationships as the shared utility both parsers call, which checks column.unique to determine cardinality.",
 "rationale": "The shared utility is the pivot point. The discrepancy must be upstream of it.",

```



```

 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cBoth the schema.rb and PostgreSQL SQL parsers call the same
`handleOneToOneRelationships` function \u2026 This function \u2026 promotes it from `ONE_TO_MANY`
to `ONE_TO_ONE` when the foreign key column has `column.unique === true`.\u201d",
 "reasoning": "The answer explicitly states that both parsers use the shared
`handleOneToOneRelationships` utility and explains that it inspects `column.unique` to determine
cardinality, directly satisfying the criterion."
 },
 {
 "id": "reasoning-48556",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Explains pg_dump separates constraints into ALTER TABLE statements instead of
inlining them in column definitions.",
 "rationale": "The root cause is a format mismatch.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cBut pg_dump does not emit uniqueness this way. A standard pg_dump output
separates constraints from table definitions: ... ALTER TABLE ONLY posts ADD CONSTRAINT
posts_user_id_key UNIQUE (user_id);\u201d",
 "reasoning": "The answer explicitly states that pg_dump separates UNIQUE constraints into ALTER
TABLE statements, satisfying the criterion."
 },
 {
 "id": "reasoning-48553",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Explains the schema.rb parser sets column.unique through extractColumnOptions when
the column definition includes unique: true.",
 "rationale": "Understanding WHY schema.rb works is half of explaining the discrepancy.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cIn the schema.rb parser
(`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to

```

``extractColumnOptions``, which reads keyword arguments from the column definition\u2026 The ``case 'unique`` branch sets ``column.unique = value instanceof TrueNode``.\u201d",

"reasoning": "The answer explicitly describes that schema.rb\u2019s parser uses

``extractColumnOptions`` to read ``unique: true`` from the column definition and set ``column.unique``, satisfying the requirement to explain how schema.rb handles the attribute."

},

{

"id": "reasoning-48559",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes `constraintToRelationship` checks `contype` against `CONSTR_FOREIGN` and returns undefined for all other constraint types.",

"rationale": "Traces the exact code that discards UNIQUE constraints.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201chandleAlterTableStmt \u2026 processes ``AT_AddConstraint`` commands by calling ``constraintToRelationship``. But that function returns early for anything except foreign keys:\n\n```\ntypescript\nif (constraint.contype !== 'CONSTR\_FOREIGN') {\n return\n ok(undefined)\n}\n```\u201d",

"reasoning": "The golden answer explicitly states that ``constraintToRelationship`` checks whether ``constraint.contype`` equals ``CONSTR_FOREIGN`` and otherwise returns undefined, showing how UNIQUE constraints are discarded."

},

{

"id": "reasoning-48554",

"tag": "reasoning",

"weight": "critical",

"description": "Explains the SQL parser only sets `column.unique` from inline `CONSTR_UNIQUE` in `ColumnDef` elements within `handleCreateStmt`.",

"rationale": "Understanding what the SQL parser DOES handle narrows the gap.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe PostgreSQL SQL parser ... handles column uniqueness inside ``handleCreateStmt``. It checks each ``ColumnDef`` element for inline ``CONSTR_UNIQUE`` or ``CONSTR_PRIMARY`` constraints\u201d",

"reasoning": "The answer clearly states that the SQL parser sets `column.unique` only when `handleCreateStmt` sees inline `CONSTR\_UNIQUE`/`CONSTR\_PRIMARY` constraints in `ColumnDef` elements, satisfying the criterion."

},

{

"id": "completeness-48573",

"tag": "completeness",

"weight": "bonus",

"description": "Locates the SQL parser test file at  
frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts.",

"rationale": "File path grounding.",

"dependent\_on": [],

"pass": true,

"evidence": "The `foreign key (one-to-one)` test in  
`frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts` declares:",

"reasoning": "The golden answer explicitly cites the SQL parser test file path, satisfying the grounding criterion."

},

{

"id": "completeness-48578",

"tag": "completeness",

"weight": "penalty",

"description": "Claims inline UNIQUE in CREATE TABLE SQL is also broken.",

"rationale": "Inline column-level UNIQUE is the one path the SQL parser handles correctly.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThis works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE constraint is part of the column definition.\u201d",

"reasoning": "The golden answer explicitly states that inline column-level UNIQUE constraints work, so it does not claim they are broken."

},

{

"id": "completeness-48577",

"tag": "completeness",

"weight": "penalty",

"description": "Claims the schema.rb parser also fails to detect column uniqueness from inline unique: true.",

"rationale": "The schema.rb parser handles this correctly.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe schema.rb parser \u2026 reads keyword arguments \u2026 When a column is declared with `unique: true` \u2026 The `case 'unique` branch sets `column.unique = value instanceof TrueNode`. \u2026 Rails' `schema.rb` format always puts column attributes inline \u2026 so `unique: true` appears right where the parser expects it.\u201d",

"reasoning": "The answer explicitly states that the schema.rb parser correctly handles inline `unique: true` attributes, so it does not claim a failure; thus the penalty criterion is satisfied."

},

{

"id": "completeness-48568",

"tag": "completeness",

"weight": "critical",

"description": "Locates handleOneToOneRelationships at frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts.",

"rationale": "File path grounds the shared utility.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cBoth the schema.rb and PostgreSQL SQL parsers call the same `handleOneToOneRelationships` function from `frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`. \u201d",

"reasoning": "The golden answer explicitly states the function name and its exact file path within the repo, satisfying the requirement to locate it."

},

{

"id": "completeness-48574",

"tag": "completeness",

"weight": "bonus",

"description": "Provides concrete SQL examples showing the inline pattern that works versus the ALTER TABLE pattern that fails.",

"rationale": "Examples make the gap tangible.",

"dependent\_on": [],

"pass": true,

"evidence": "Works: \"This works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE constraint is part of the column definition.\" Fails: \"But `pg\_dump` does not emit uniqueness this way... `ALTER TABLE ONLY` posts ADD CONSTRAINT posts\_user\_id\_key UNIQUE (user\_id);\"",

"reasoning": "The golden answer explicitly contrasts an inline `CREATE TABLE ... UNIQUE` example that succeeds with an `ALTER TABLE ... ADD CONSTRAINT ... UNIQUE` example that fails, satisfying the criterion for concrete SQL examples showing the working and failing patterns."

},

{

"id": "completeness-48571",

"tag": "completeness",

"weight": "critical",

"description": "Notes no SQL parser test exercises the scenario where uniqueness is added via ALTER TABLE rather than inline on the column.",

"rationale": "The prompt explicitly asks why the test suite doesn't catch this.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe SQL parser's one-to-one test \u2026 declares \u2026 UNIQUE is inline on the column definition\u2026 The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself.\u201d",

"reasoning": "The answer explicitly explains that both SQL parser tests keep UNIQUE inline and do not cover the ALTER TABLE-added uniqueness scenario, fulfilling the requirement."

},

{

"id": "completeness-48572",

"tag": "completeness",

"weight": "critical",

"description": "Notes the FIXME comment and skip'd test in the SQL parser test file about CONSTRAINT statement support.",

"rationale": "Documentary evidence the parser creators knew about the limitation.",

"dependent\_on": [],

"pass": true,

"evidence": "The test file also contains a skip'd test with a FIXME comment (`// FIXME: 'CONSTRAINT' statement is not supported yet`) for table-level CONSTRAINT syntax inside CREATE TABLE.",

"reasoning": "The golden answer explicitly notes the skip'd test and its FIXME comment in the SQL parser test file about CONSTRAINT statement support, satisfying the criterion."

},

```

{
 "id": "completeness-48567",
 "tag": "completeness",
 "weight": "critical",
 "description": "Locates the schema.rb parser at
frontend/packages/db-structure/src/parser/schemarb/parser.ts.",
 "rationale": "File path grounds the schema.rb parser analysis.",
 "dependent_on": [],
 "pass": true,
 "evidence": "In the schema.rb parser
(`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to
`extractColumnOptions`...",
 "reasoning": "The golden answer explicitly identifies the schema.rb parser's location at the required file
path."
},
{
 "id": "completeness-48570",
 "tag": "completeness",
 "weight": "critical",
 "description": "Identifies constraintToRelationship as the SQL converter function that filters out non-FK
constraints.",
 "rationale": "Names the specific function where UNIQUE constraints are discarded.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The `ALTER TABLE` statement goes through `handleAlterTableStmt`, which processes
`AT_AddConstraint` commands by calling `constraintToRelationship`. But that function returns early for
anything except foreign keys: `if (constraint.contype !== 'CONSTR_FOREIGN') { return ok(undefined) }`.",
 "reasoning": "The golden answer explicitly identifies `constraintToRelationship` as the function invoked
by the SQL parser for ALTER TABLE constraints and explains that it filters out non-foreign-key constraints,
causing UNIQUE constraints to be discarded."
},
{
 "id": "completeness-48566",
 "tag": "completeness",
 "weight": "critical",

```

```

 "description": "Locates the SQL converter at
frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts.",
 "rationale": "File path grounds the SQL parser analysis.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe PostgreSQL SQL parser
(`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`) handles column
uniqueness\u2026\u201d",
 "reasoning": "The golden answer explicitly identifies the SQL converter file path, grounding its analysis
in the required location."
 },
 {
 "id": "completeness-48576",
 "tag": "completeness",
 "weight": "bonus",
 "description": "Notes handleIndexStmt stores index data in table.indexes without updating
column.unique.",
 "rationale": "Clarifies index data exists but is disconnected from column metadata.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cCREATE UNIQUE INDEX statements are captured by `handleIndexStmt` as Index
objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique`
field.\u201d",
 "reasoning": "The golden answer explicitly states that `handleIndexStmt` records index uniqueness
separately without updating `column.unique`, directly satisfying the criterion."
 },
 {
 "id": "completeness-48569",
 "tag": "completeness",
 "weight": "critical",
 "description": "Identifies extractColumnOptions as the schema.rb parser function that reads the unique
keyword from column definitions.",
 "rationale": "Names the specific function in the schema.rb parser responsible for the working path.",
 "dependent_on": [],
 "pass": true,

```

```

 "evidence": "\u201cIn the schema.rb parser
(frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to
`extractColumnOptions`, which reads keyword arguments from the column definition \u2026 The `case
'unique` branch sets `column.unique = value instanceof TrueNode`.\u201d",
 "reasoning": "The golden answer explicitly names `extractColumnOptions` as the schema.rb parser
function that processes column options and handles the `unique` keyword, satisfying the criterion."
 },
 {
 "id": "completeness-48575",
 "tag": "completeness",
 "weight": "bonus",
 "description": "Notes table-level UNIQUE constraints in CREATE TABLE are also skipped because
handleCreateStmt only processes ColumnDef elements.",
 "rationale": "A third distinct gap.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe same gap exists for two other SQL patterns. Table-level UNIQUE constraints
inside CREATE TABLE (`UNIQUE (user_id)` as a separate clause) are skipped because `handleCreateStmt`
only processes elements where `ColumnDef` in elt`.\u201d",
 "reasoning": "The golden answer explicitly states that table-level UNIQUE constraints in CREATE TABLE
are skipped and explains that this happens because handleCreateStmt only processes ColumnDef
elements, meeting the criterion."
 },
 {
 "id": "style-48581",
 "tag": "style",
 "weight": "bonus",
 "description": "Avoids proposing a complete code patch, staying focused on diagnosis.",
 "rationale": "RCA intent is diagnosis, not implementation.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The answer describes existing code behavior and shortcomings without proposing any
new code changes or patch-style solutions.",
 "reasoning": "It stays focused on diagnosis, explaining how current parsers work and where they fail,
without suggesting implementation steps or code modifications."
 },
}

```



```

{
 "id": "style-48580",
 "tag": "style",
 "weight": "bonus",
 "description": "References specific file paths, function names, and code patterns from the repository.",
 "rationale": "Repo-grounded answers are more verifiable.",
 "dependent_on": [],
 "pass": true,
 "evidence": "Mentions files like
`frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`,
`frontend/packages/db-structure/src/parser/schemarb/parser.ts`, and
`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`, as well as functions
(`handleOneToOneRelationships`, `extractColumnDetails`, `handleCreateStmt`, etc.) and specific code
snippets from those locations.",
 "reasoning": "The answer references particular repository files, functions, and code snippets, showing
how unique constraints are parsed differently in schema.rb versus SQL parsing, fulfilling the
repo-grounded criterion."
},
{
 "id": "style-48579",
 "tag": "style",
 "weight": "bonus",
 "description": "Traces both parsers systematically, comparing their uniqueness handling paths.",
 "rationale": "A cross-parser comparison is clearer when both sides are traced explicitly.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cHow the schema.rb parser sets column.unique\u2026 Rails\u2019 schema.rb
parser reads these inline attributes through extractColumnOptions\u2026; How the SQL parser fails to set
column.unique\u2026 The PostgreSQL SQL parser\u2026 handles column uniqueness inside
handleCreateStmt\u2026 The UNIQUE constraint is dropped. `column.unique` stays `false`.\u201d",
 "reasoning": "The answer explicitly walks through both parsers\u2019 code paths, detailing how
schema.rb sets `column.unique` while the SQL parser misses it, thereby systematically comparing their
uniqueness handling."
}
],
"passed": 30,

```

```
"failed": 0,
"total": 30,
"score_percent": 100.0,
"summary": "Passed 30/30 criteria (100.0%)"
}
```

## Test Model

### Model Response

```
=====
RUN 1/3
```

```
=====

In the PostgreSQL SQL parser (see
`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`), the only place a column's
`unique` flag is ever set is inside `handleCreateStmt`. When a `ColumnDef` node is processed, the code
looks only at that column's inline constraints:
```

```
```ts
columns[colDef.colname] = {
  ...
  unique:
    colDef.constraints
      ?.filter(isConstraintNode)
      .some((c) =>
        ['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(
          c.Constraint.contype ?? "",
        ),
      ) || false,
  ...
}
```
```

(lines 178-186)

So a column ends up marked `unique: true` **only if** its own `ColumnDef` contains `UNIQUE` or `PRIMARY KEY` directly in the column definition (mention `TEXT UNIQUE`, `user_id INT PRIMARY KEY`, etc.). That's why the parser's "one-to-one" test (which uses the inline form `user_id INT REFERENCES users(id) UNIQUE`; see `index.test.ts` around lines 183-194) passes—the inline constraint is converted into `colDef.constraints` and triggers the check above.

Any `UNIQUE` constraint that appears outside the column definition is ignored for column metadata:

- \* Table-level constraints such as `CONSTRAINT users_email_key UNIQUE (email)` or `UNIQUE (email)` are emitted by `pg_dump`, but they show up as separate `Constraint` entries in the table element list, not inside `ColumnDef`, and there is no code that maps them back to the referenced column.

- \* Likewise, `CREATE UNIQUE INDEX ...` statements are parsed by `handleIndexStmt` (lines 227-265) to populate the table's `indexes`, but that code never updates `columns[col].unique`.

Because `handleOneToOneRelationships` (lines 3-15 in `parser/utils/handleOneToOneRelationships.ts`) decides whether a relationship is `ONE_TO_ONE` by checking `foreignColumn?.unique`, only relationships whose foreign key column had an inline `UNIQUE`/`PRIMARY KEY` constraint will switch to one-to-one. Everything driven by table-level `UNIQUE` constraints (the usual `pg_dump` output) will keep `unique: false` on the column and therefore stay `ONE_TO_MANY`.

So, in short: the parser currently "picks up" only inline column constraints (`colname ... UNIQUE/PRIMARY KEY`). `UNIQUE` constraints expressed via table constraints or indexes are not reflected in the column metadata, which is why your `pg_dump` output isn't affecting the one-to-one detection.

=====

RUN 2/3

=====

In the PostgreSQL SQL parser (`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`) the only place where the `Column.unique` flag is set is inside the per-column loop of `handleCreateStmt`. For each `ColumnDef`, it looks at the inline constraints that belong to that column and sets:

```
```ts
unique: colDef.constraints
  ?.filter(isConstraintNode)
  .some((c) =>
```

```
['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(c.Constraint.contype ?? ")  
) || false,  
...
```

(see roughly lines 150-190 of that file). In other words, a column ends up marked as ``unique: true`` only when the ORIGINAL table definition includes an inline constraint such as:

```
```sql  
mention TEXT UNIQUE
-- or
user_id INT REFERENCES users(id) UNIQUE
...
```

This is the pattern covered by the unit test “foreign key (one-to-one)” in ``frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts`` (around lines 180-210). Because the UNIQUE is inline, ``colDef.constraints`` contains ``CONSTR_UNIQUE``, so the ``unique`` flag is set and ``handleOneToOneRelationships`` later upgrades the relationship to ``ONE_TO_ONE``.

However, `pg_dump` normally emits UNIQUE constraints as table-level constraints or ALTER TABLE statements, for example:

```
```sql  
ALTER TABLE ONLY public.users  
ADD CONSTRAINT users_email_key UNIQUE (email);  
...
```

or as standalone ``CREATE UNIQUE INDEX ...`` commands. Those statements are parsed into either ``Constraint`` nodes that are siblings of the columns (not part of any ``ColumnDef``) or ``IndexStmt`` nodes. The current converter ignores both with respect to column metadata:

* Table-level ``Constraint`` nodes are never inspected when building ``columns``. Only ``ColumnDef`` entries are processed (see the ``for (const elt of createStmt.tableElts)`` loop).

* Unique ``IndexStmt``s are stored under ``table.indexes``, but nothing propagates their uniqueness back onto the participating columns.

The ALTER TABLE handler (`handleAlterTableStmt``) even filters exclusively for foreign key additions; it doesn't look at UNIQUE constraints at all.

Therefore the deciding factor is **how the UNIQUE constraint appears in the SQL**. Inline column constraints are picked up; anything expressed as a table-level constraint, an ALTER TABLE ADD CONSTRAINT, or a CREATE UNIQUE INDEX is currently ignored by the column metadata, so relationships whose uniqueness is enforced that way never see `foreignColumn.unique = true``.

To make pg_dump output work, you'd need to extend the converter to (a) process table-level `Constraint`` nodes of type `CONSTR_UNIQUE`/CONSTR_PRIMARY``, (b) process UNIQUE constraints added in ALTER TABLE commands, and/or (c) infer column uniqueness from unique indexes. Until then, only inline UNIQUE syntax will be reflected in the parser's column metadata.

```
=====
RUN 3/3
```

```
=====
The SQL parser decides whether a column is "unique" solely inside convertToSchema` when it walks each ColumnDef` inside a CREATE TABLE ...` statement (see frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`, lines 132-196). For every column it sets column.unique = true` only if one of that column's inline constraints has contype === 'CONSTR_UNIQUE'` (or CONSTR_PRIMARY`). No other place in the parser ever flips that flag.
```

What the parser does **not** do right now:

- * It never looks at table-level `Constraint`` nodes that appear alongside `ColumnDef`` entries in `CreateStmt.tableElts``. That means `UNIQUE (user_id)`` or `CONSTRAINT users_user_id_key UNIQUE (user_id)``—the pattern `pg_dump`` prefers—are ignored entirely, so the column metadata for `user_id`` keeps `unique: false``.

- * It never revisits columns when it later processes `CREATE UNIQUE INDEX ...`` or `ALTER TABLE ... ADD CONSTRAINT ... UNIQUE ...``. `handleIndexStmt`` (lines 227-265) just records the index object; it doesn't propagate uniqueness back to the column. And `handleAlterTableStmt`` (lines 327-360) only cares about foreign keys.

Because `handleOneToOneRelationships`` (`frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts``, lines 4-13) upgrades a relationship to `ONE_TO_ONE`` **only when** the foreign column already has `unique: true``, the feature “works” in the unit tests (they all declare `user_id INT ... UNIQUE`` inline) but fails for `pg_dump``-style uniqueness (table constraints or separate indexes). In short, the parser picks up UNIQUE constraints only when they are written inline in the column definition inside `CREATE TABLE``; any UNIQUE declared elsewhere is currently ignored, which is why some of your `pg_dump` constraints never show up in the column metadata.

Evaluation JSON

```
{
  "points": 24.0,
  "maxPoints": 53.0,
  "score_percent_weight": 45,
  "summary_weight": "24.0/53.0 points (45%)",
  "eval_runs": 3,
  "eval_runs_succeeded": 3,
  "per_run_points": [
    23.0,
    24.0,
    25.0
  ],
  "results": [
    {
      "id": "reasoning-48560",
      "weight": "bonus",
      "pass": true,
      "points": 1.0
    },
    {
      "id": "reasoning-48557",
      "weight": "critical",
      "pass": true,
      "points": 3.0
    },
  ],
}
```

```
{
  "id": "reasoning-48558",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-48561",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48562",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48555",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48565",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48563",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "reasoning-48564",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48552",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "reasoning-48556",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48553",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48559",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48554",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
}
```



```
{
  "id": "completeness-48573",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48578",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48577",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48568",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-48574",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48571",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "completeness-48572",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48567",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48570",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48566",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-48576",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48569",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "completeness-48575",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48581",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48580",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48579",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
}
]
```

model eval score 45.3

difficulty medium